```
// index.php
print $twig->render('block-use.html.twig', [
    'parent' => 'parent.html.twig'
]);
```

```
{# views/block-use.html.twig #}
{% extends parent %}
{% use 'inc/blocks.html.twig' %}
{% block titleEle %}
  <title>{{ '{%' }} use {{ '%}' }} DEMO</title>
{% endblock %}
```

```
{# views/inc/blocks.html.twig #}
{% block bodyContent %}
  <p>Content from blocks.html.twig</p>
{% endblock %}
```

When we put this all together, we get the following content rendered:

```
$ curl -v http://localhost:8080/index.php
<!DOCTYPE html>
<html>
  <head>
    <title>{% use %} DEMO</title>
  </head>
  <body>
    <p>Content from blocks.html.twig</p>
  </body>
</html>
```

The `{% use 'inc/blocks.html.twig' %}` tag tells Twig to read in the specified template and treat all `block` tags as if they exist in the calling template. This opens up even more strategies for organizing your templates into reusable units. I've also taken the opportunity to demo a couple of other features: the `{% extends %}` tag in `block-use.html.twig` is using a variable rather than a string literal (and will take any other Twig expression), and the `titleEle` block in that template is using `{{ '{%' }}` string literals to escape Twig syntax (which is the recommended method).